



KTH Technology and Health

TENTAMEN

Kurs, kursnummer	HI1027, Objektorienterad programmering
Moment:	TEN1, 3,5 hp
Program: Åk:	TIDAA, TIELA, TIMEL -
Examinator:	Anders Lindström
Rättande lärare:	Anders Lindström
Datum: Tid:	Tisdag 2023-03-14 08:00-13:00
Hjälpmedel:	IntelliJ och Netbeans finns på tentamenskontot API-dokumentation och en kort syntaxlista finns i mappen P/Java/api/index.html
Omfattning och betygsgränser:	För betyget E krävs minst hälften av poängen på A-delen samt och hälften av poängen på B-delen samt att minst en av de två första programmeringsuppgifterna är helt korrekt lösta. B-delen rättas endast om minst hälften av poängen på A-delen uppnåtts. För betyg C krävs totalt minst 70 % av totala poängen; för betyg A krävs minst 90 % av totala poängen.
Övrig information:	Del A, Teoriuppgifter: Svara på separat papper. Del B, Programmeringsuppgifter: Lösningar lämnas på tentamenskontot. <i>När du skapar projektet i IntelliJ eller NetBeans måste du kontrollera att projektet hamnar under enheten H, på ditt tentamenskonto. Alla lösningar skrivs i ett och samma projekt, men varje uppgift ska ligga i ett separat paket, med namnen uppg 9, uppg10, ...</i> Innan du avslutar tentamen: Kontrollera att du verkligen sparar dina lösningar under H

Mac-användare: För att komma åt måsvingar, hakparenteser och "|", använd "AltGr" + lämplig tangent, t.ex. "AltGr" + "{" (7:ans tangent).

Del A, Teoriuppgifter

Svara, på separat papper, kort men koncist på frågorna. På fritextfrågor ska svaren motiveras. Otydliga lösningar, både vad gäller läsbarhet och begriplighet, rättas inte.

1. Ange för frågorna a respektive b vilket alternativ som är korrekt (endast ett alternativ per delfråga).
 - a) Vilken synlighet har, i Java, en datamedlem eller metod som deklarerats `protected`?
 - b) Vilken synlighet har, i Java, en datamedlem eller metod som varken deklarerats `public`, `protected` eller `private`?
 - A. Synlig endast i klassen där den deklarerats
 - B. Synlig i klassen samt i andra klasser i samma paket
 - C. Synlig klassen samt i subklasser
 - D. Synlig i klassen, i subklasser samt i andra klasser i samma paket(1+1 p)

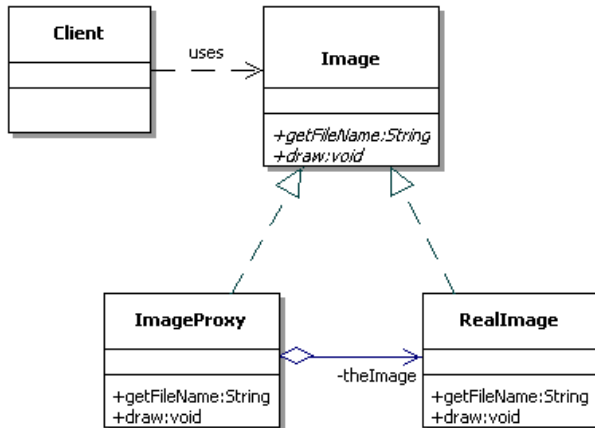
I uppgift 2–4 ska du ange vilka alternativ (noll till flera) som är korrekta.

2. I en abstrakt klass får, i språket Java, följande ingå:
 - A. Konstruktörer
 - B. Datamedlemmar
 - C. Klassdata
 - D. Metoder med implementation
 - E. Abstrakta metoder(2 p)
3. Antag att en metod, `foo`, implementeras i en klass `Super`, och omdefinieras i en subklass, `Sub`. Ett objekt skapas med satsen `Super superRef = new Sub();` Vad är korrekt?
 - A. `superRef.foo();` innebär att superklassens version av metoden `foo` exekveras
 - B. Subklassens version av `foo` kan, om så behövs, anropa superklassens version
 - C. Det går, under exekveringen, inte att via programkod undersöka vilken typ av objekt referensen `superRef` refererar
 - D. Vid omdefinieringen av `foo` i subklassen får antalet argument, och deras typ, ändras förutsatt att metodens namn och returtyp inte ändras(2 p)
4. Vilka av nedanstående påståenden är korrekta vad gäller begreppet hög kohesion?
 - A. Hög kohesion innebär att alla datamedlemmar måste vara inkapslade.
 - B. Hög kohesion innebär bland annat att interna ändringar i klassen med hög kohesion har minimal påverkan på andra klasser/moduler.
 - C. Hög kohesion innebär bland annat att en klass/modul ska modellera en och endast en sak i problemområdet.
 - D. Att programmera mot gränssnitt, s.k. `abstract coupling`, är ett exempel på hög kohesion.
 - E. Det är lättare att förstå hur en klass ska implementeras och testas om den har hög kohesion.(2 p)

5. Se klassdiagrammet nedan.

a) Vad innebär symbolen mellan Image och ImageProxy? Hur "realiseras" detta vid implementationen av klassen ImageProxy?

b) Vad innebär symbolen mellan klassen ImageProxy och RealImage? Hur "realiseras" detta vid implementationen av klassen ImageProxy?



(2 p)

6. Förklara till vad, och hur, kod-block märkta med nyckelordet finally används vid felhantering. Skriv ett enkelt, och rimligt, kodexempel som visar hur finally kan används inuti en metod, utan att felet, exception, fångas i nämnda metod.

Koden relaterad till finally, och tillhörande syntax, måste vara korrekt men det som utförs inuti respektive kodblock kan beskrivas översiktligt eller med pseudokod.

(2 p)

7. Beskriv designmönstret Composite. Ange också ett konkret exempel på en situation där detta designmönster kan användas.

(2 p)

8. Antag att vi har skrivit en list-klass, som vi försökt göra trådsäker genom att synkronisera alla metoder som läser från/skriver till ändringsbara datamedlemmar, se nedan.

```
public class List {
    ...
    public synchronized T get(int index) {...}
    public synchronized void add(T object) {...}
    public synchronized Iterator<T> getIterator() {...}
}
```

Antag att vi skapar ett objekt av list-typen ovan och vill iterera över detta. Visa hur ett sådant kodstycke ska se ut för att även detta kodstycke ska vara trådsäkert.

(2 p)

Del B, Programmeringsuppgifter

Alla klasser i denna del av tentamen ska följa objektorienterade principer, speciellt inkapsling. Om inte annat anges ska alla klasser ha meningsfulla konstruktorer, som korrekt initierar alla datamedlemmar, access-metoder samt en omdefinierad version av toString.

Lösningar lämnas i ett gemensamt projekt på tentamenskontot. För varje lösning ska det finnas ett paket med namnet "uppg9", "uppg10" etc. Kontrollera att du verkligen sparar dina lösningar på H. Till dessa uppgifter kan du i vissa fall ha hjälp av Javas API-dokumentation, som finns på P:/Java/api.

9. Fordon

I denna uppgift behöver du en klasserna Person och Gender som finns på P/Java/HI1027.

Skriv en abstrakt klass, Vehicle, som representerar det som är gemensamt för olika typer av fordon. Ett fordon har ett registreringsnummer (en textsträng), en ägare (typen Person) samt en färg (en enum som du själv definierar). *Alla datamedlemmar ska vara privata.*

Endast ägaren ska kunna ändras. Konstruktorn ska kasta ett IllegalArgumentException om argumentet för registreringsnumret inte är exakt sex tecken långt.

Klassen Vehicle ska även ha en abstrakt metod, getLicenseClass, som returnerar ett enum-värde av typen LicenseClass (körkortsklass). Du skriver själv denna enum, som ska ha värdena A, A1 och B.

Skriv sedan nedanstående två subklasser.

Klassen Car representerar en bil. En bil har, förutom ärvt data, en datamedlem som representerar antalet dörrar (kan ej ändras) och tillhörande access-metod.

Metoden getLicenseClass ska implementeras så att den alltid returnerar LicenseClass.B.

Klassen Motorcycle ska ha en icke ändringsbar datamedlem, boolean isLightWeight, som representerar om motorcykeln är lättviktig eller ej. Metoden getLicenseClass ska implementeras så att den returnerar LicenseClass.A1 om motorcykeln är lättviktig, i annat fall A.

Båda subklasserna ska ha korrekta implementationer av toString som även presenterar ärvt data.

Skriv en main-metod som skapar en lista med Vehicle-referenser och lägger till objekt av de olika subtyperna. Kontrollera att anrop getLicenseClass ger korrekta resultat.

(4 p)

10. Att göra

Skriv en klass, ToDoItem, som representerar något som ska utföras. Klassen ska datamedlemmar för titel samt datum då uppgiften ska vara avklarad (dueDate, typen LocalDate).

Titeln ska ej kunna ändras, men datumet ska kunna ändras. Det ska inte vara möjligt att sätta ett datum som redan passerat, ett IllegalArgumentException ska kastas om det görs.

Klassen ska implementera det generiska interfacet Comparable på så sätt att datumet jämförs, men om dessa är lika ska i stället titeln jämföras. Metoden equals, från klassen Object, ska omdefinieras så den returnerar true om, och endast om, compareTo returnerar 0.

Skriv sedan klassen `ToDoList` som representerar en samling `ToDoItems`. Du får använda valfri listklass internt, men det är viktigt att denna lista endast kan manipuleras via klassens metoder. Kontakt för `ToDoList`: *Objekten i listan ska alltid ligga i den ordning som definieras av `ToDoItem.compareTo`.*

Klassen publika gränssnitt ska (endast) ha nedanstående funktionalitet.

- En parameterlös konstruktor samt metoden `size()`, som returnerar antal items.
- `addToDo(title, dueDate)`, som skapar ett nytt `ToDoItem` och lägger detta i den interna listan; platsen bestäms av rangordningen som definieras av `ToDoItem.compareTo`.
- `removeNextToDo()`, som returnerar det element som står näst i tur och tar bort detta från listan. Om listan är tom ska ett `IllegalStateException` kastas.
- `postponeNextToDo(newDueDate)`, som uppdaterar `dueDate` för det objekt som står näst i tur. Notera att detta kan innebära att objektet ska få en annan plats i listan.
- `toString`

Skriv slutligen en `main`-metod som demonstrerar funktionaliteten.

(4 p)

11. Lista

I denna uppgift får du *inte* använda dig av färdigdefinierade samlings-klasser som `ArrayList`, `LinkedList` eller liknande.

Du ska implementera en *generisk* list-klass, `AList`, där objekten lagras i en intern och privat array. Klassen publika gränssnitt ska (endast) ha nedanstående funktionalitet.

- Två konstruktorer, en parameterlös som skapar en array av storlek 3 och en som tar arrayens storlek som argument,
- `size()`, som returnerar aktuellt antal element.
- `get(int index)`, som returnerar en referens till aktuellt objekt.
- `add(int index, T obj)`, som placerar ett nytt element på angiven position i listan. Objektet från och med angiven position ska flyttas ett steg mot slutet av arrayen. Om elementen inte får plats i arrayen ska denna utökas.
- `boolean remove(T obj)`, som, om `obj` finns i listan, tar bort `obj` ur listan och returnerar `true`. Om objektet inte existerar i listan returneras `false`. Använd `equals` vid jämförelsen.
- `clear()`, som tömmer listan.
- `getCopy()`, som returnerar en kopia av list-objektet, med en egen array. En grund kopia räcker.

Metoderna `clear` och `remove` ska sätta positioner där objekt "tagits bort" ur arrayen till `null`. Metoder som tar ett index som argument ska kasta `IndexOutOfBoundsException` om index är felaktigt.

Skriv sedan en `main`-metod som skapar ett list-objekt och demonstrerar funktionaliteten.

(4 p)

12. Funktionellt

Du ska skriva funktionalitet för att utifrån en lista med objekt skapa en ny lista, av samma storlek, där varje objekt i den nya listan representerar något hos motsvarande objekt i den ursprungliga listan. Exempelvis ska vi kunna anropa metoden med en lista av Person-objekt och få en lista med personernas namn.

För detta behöver du skriva ett interface, IMapper, med en metod, apply, som tar ett objekt som argument och returnerar ett nytt objekt, eller värde. Interfacet ska vara generiskt och behöver två typparametrar, den ena representerar argumentets typ och den andra representerar typen på det som returneras.

Metoden för att konvertera en hel lista till ett enda värde, map, skrivs förslagsvis som en klassmetod, i en separat klass, med nedanstående signatur.

```
public static <A, R> List<R> map(IMapper<A, R> mapper, List<A> elements)
```

Skapa sedan de tre implementationer av interfacet som beskrivs nedan.

1. NameMapper, som tar ett Person-objekt och returnerar ett String-objekt med personens namn. Klassen Person är den samma som i uppgift 9.
2. ToUpperCaseMatcher som tar String-objekt och returnerar ett nytt objekt, med texten omvandlad till versaler.
3. IsFemaleMapper som tar ett Person-objekt och returnerar ett boolskt värde som representerar om personen är en kvinna eller ej. Använd wrapper-klassen Boolean för att representera returvärdet.

Skriv en main-metod som demonstrerar funktionaliteten genom att anropa metoden map med respektive interface-implementation.

Du kan, om du vill, i något av de tre fallen ovan skriva ett lambda-uttryck som representerar interface-implementationen (ej ett krav).

(4 p)